Reed ME:

Before I mention the patching notes, I need to note that the code files that I used are from my submission of lab #3. To run the system after I applied the patches of lab 3 follow the following steps:

- Load the password of the server to the OS:
  - In Linux cmd: export SERVER_PASSWORD='!Q#E%T&U8i6y4r2w'
  - In Windows: $env: SERVER_PASSWORD="!Q#E%T&U8i6y4r2w"
- Load the encryption/decryption key to the OS:
  - In Linux: export NETWORK_KEY='1234567890abcdef'
  - In Windows: $env: NETWORK_KEY=”1234567890abcdef”

Patch Note of Vulnerability #1: SQL Injection

In **app.py**, specifically at **login()** function, if the user enters a wrong password, the code takes the exact argument inserted and concatenate it in SQL query. The result of the query is returned as it is in the error message. An attacker can inject malicious SQL statements such as “ ' OR '1'='1 “ to manipulate the SQL query. This may cause unintended database behavior or expose information such as user ID, password, or tokens when the application returns an error message.

```
    db_message = ''
    db_query = "SELECT * FROM users WHERE password = '" + password + "'"
    db_password = conn.execute(db_query).fetchone()
    for x in db_password:
        db_message = db_message + ":" + str(x)
    return render_template('login.html', Err=str(db_message))
xcept Exception as ex:
    return render_template('login.html', Err=ex)
eturn render_template('login.html', Err="Wrong password")
```

So, instead of returning the result of the database query, we will simply return error message says “Wrong password” as follows:

```
        return render_template('authenticate.html', Temp=0.00, Token=1
    else:
        return render_template('login.html', Err="Wrong password")
```

Patch Note of Vulnerability #2: No Rate limiting in case of using wrong password

In **app.py**, specifically at **login()** function, if the user enters wrong password multiple times, the application continues to process them without restriction. This helps attackers when they are practicing brute force attack. But, if there is rate limiting technique, bruit force attack will not be feasible.

The code before patching the vulnerability:

```
@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method=='GET': # if the request is a GET we return the login page
        return render_template('login.html')
    else:
        conn = get_db_connection()
        password = request.form.get('authToken')
        is_password, act_token = verify_password(conn, password)

        if is_password:
            snc = SampleNetworkClient.SimpleNetworkClient(23456, 23457)
            is_auth = snc.authenticate(23456, bytes(act_token, 'utf-8'))
            return render_template('authenticate.html', Temp="0.00", Token=is_auth.decode('utf-8'))
        else:
            return render_template('login.html', Err="Wrong password")
```

I patch the vulnerability by introducing many functions to support the rate limiting. To start, the first function is **failedLogin()**. This function creates a record for the user who attempts to login unsuccessfully, the record based on the user IP. Note that it is not the best practice to locked the account based on IP, but for the current design there is no user ID. Then we will count how many times the user tried to login unsuccessfully. If the user reached the maximum attempts, which I decided to be three times in my patch, we will increase the red flag level for the user by one. Alos, we will set time to lock, which I decided to be 60 seconds for the first lock, and it multiplied by red flag level in the next failure attempt. For example, If the user locked for the first time, they need to wait for 60 second, and 120 second for the next time and so on.

```
def failedLogin(ip):
  current_time= time.time()
  if ip not in login_failuer:
    login_failuer[ip] = {"count" : 0, "level" : 0, "locked_until" :0} # create record for the user who attemp to login unsucessfully

  login_failuer [ip]["count"] +=1

  if login_failuer[ip]["count"] >= max_attempt:
    login_failuer[ip]["level"] +=1
    time_to_lock= lock_time * login_failuer[ip]["level"]
    login_failuer[ip]["locked_until"] = current_time + time_to_lock
```

Furthermore, I created **isLocked()** function, which will be used in **login()** function to check if the user is locked out or not. In addition, it returns the remaining time, if the user was locked.

```
def isLocked (ip):
  if ip in login_failuer:
    data = login_failuer[ip]
    if time.time() < data ["locked_until"]:
      data = login_failuer[ip]
      remaining_time = int(data["locked_until"] - time.time())
      if remaining_time > 0:
        return True, remaining_time

  return False, 0
```

The final function that I created is **resetFailuer()**, which deletes the user record from login_failuer when the user login successfully.

```
def resetFailuer(ip):
  if ip in login_failuer:
    del login_failuer[ip]
```

Finally, I updated the **login()** function to check if the user is locked or not. If the user locked, it returns the remaining time to retry. Also, when the user tried to login unsuccessfully it will trigger **failedLogin()**. Finally, resetFailuer() will be triggered if the user entered the correct password.

```
def login():
    if request.method=='GET': # if the request is a GET we return the login page
        return render_template('login.html')
    else:
        ip = request.remote_addr ## get the ip for the user attempt to login
        is_locked, remaining_time = isLocked(ip)
        if is_locked:
            return render_template('login.html', Err=f"Account is locked. Try again in {remaining_time} seconds.")
        conn = get_db_connection()
        password = request.form.get('authToken')
        is_password, act_token = verify_password(conn, password)

        if is_password:
            resetFailuer(ip) ## if the user login successfully, remove them from login_failuer
            snc = SampleNetworkClient.SimpleNetworkClient(23456, 23457)
            is_auth = snc.authenticate(23456, bytes(act_token, 'utf-8'))
            return render_template('authenticate.html', Temp="0.00", Token=is_auth.decode('utf-8'))
        else:
            failedLogin(ip)
            return render_template('login.html', Err="Wrong password")
```

Patch Note of Vulnerability #3: Missing exception handling in DB connection

In **app.py**, specifically at **get_db_connection()** function, it connects to the database without error handling. If anything went wrong with the connection such as access issue, the system would crash and affect the availability of system. In addition, the code never closes the database connection, which is also considered as vulnerability.

The code before patching the vulnerability:

```
def get_db_connection():
    conn = sqlite3.connect('database.db')
    conn.row_factory = sqlite3.Row
    return conn
```

```
def login():
    if request.method=='GET': # if the request is a GET we return the login page
        return render_template('login.html')
    else:
        ip = request.remote_addr ## get the ip for the user attempt to login
        is_locked, remaining_time = isLocked(ip)
        if is_locked:
            return render_template('login.html', Err=f"Account is locked. Try again in {remaining_time} seconds.")
        conn = get_db_connection()
        password = request.form.get('authToken')
        is_password, act_token = verify_password(conn, password)

        if is_password:
            resetFailuer(ip) ## if the user login successfully, remove them from login_failuer
            snc = SampleNetworkClient.SimpleNetworkClient(23456, 23457)
            is_auth = snc.authenticate(23456, bytes(act_token, 'utf-8'))
            return render_template('authenticate.html', Temp="0.00", Token=is_auth.decode('utf-8'))
        else:
            failedLogin(ip)
            return render_template('login.html', Err="Wrong password")
```

For patching these two vulnerabilities, I added error handling at **get_db_connection()** function that return **conn** if the sqlite3 connect to the database successfully, otherwise it will return **None** as follows:

```
def get_db_connection():
    try:
      conn = sqlite3.connect('database.db')
      conn.row_factory = sqlite3.Row
      return conn
    except sqlite3.Error:
      return None
```

In **login()** function, I added a condition that return database connection failed message if the connection had an issue. In addition, I closed the connection after the code finished with the database connection. The code after patching is as follows:

```
@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method=='GET': # if the request is a GET we return the login page
        return render_template('login.html')
    else:
        ip = request.remote_addr ## get the ip for the user attempt to login
        is_locked, remaining_time = isLocked(ip)
        if is_locked:
            return render_template('login.html', Err=f"Account is locked. Try again in {remaining_time} seconds.")
        conn = get_db_connection()
        if conn is None:
            return render_template('login.html', Err="Database connection failed")
        password = request.form.get('authToken')
        try:
            is_password, act_token = verify_password(conn, password)
        except Exception:
            conn.close()
            return render_template('login.html', Err="Database error!")

        if is_password:
            resetFailuer(ip) ## if the user login successfully, remove them from login_failuer
            snc = SampleNetworkClient.SimpleNetworkClient(23456, 23457)
            is_auth = snc.authenticate(23456, bytes(act_token, 'utf-8'))
            conn.close()
            return render_template('authenticate.html', Temp="0.00", Token=is_auth.decode('utf-8'))
        else:
            failedLogin(ip)
            return render_template('login.html', Err="Wrong password")
```

Patch Note of Vulnerability #4: Multiple command process at one authentication request

In **SampleNetworkServer.py**, specifically **processCommands()** function it takes a message which contains the authentication login and the commands. The issue is that the rest of the message may contain many commands separated by semicolon, the code does not check. For security reasons, there should be authentication between every command. Because maybe the first command is authorized for the requester, but the second one is not authorized. Attackers may bypass the authorization by sending authorized command first just to open the door for the second command.

The code before patching the vulnerability:

```
def processCommands(self, msg, addr) :
    cmds = msg.split(';')
    for c in cmds :
        cs = c.split(' ')
        if len(cs) == 2 : #should be either AUTH or LOGOUT
            if cs[0] == "AUTH":
                self.delExpiredToken() ##We check and delete expired token every time we need to authenticate
                if cs[1] == SERVER_PASSWORD :
                    if len(self.tokens) >= MAX_TOKENS :
                      oldest_token = min(self.tokens, key=self.tokens.get) ## Delete the oldest token if the token list is full
                      del self.tokens[oldest_token]
                    newToken=''.join(random.choice(string.ascii_uppercase + string.ascii_lowercase + string.digits) for _ in range(16))
                    self.tokens[newToken] = time.time() ## Add the new token to the token list
                    self.serverSocket.sendto(self.encrypt(newToken), addr)
                    #print (self.tokens[-1])
                else:
                    self.serverSocket.sendto(self.encrypt("Incorrect password"), addr)
            elif cs[0] == "LOGOUT":
                if cs[1] in self.tokens :
                    del self.tokens[cs[1]]
            else : #unknown command
                self.serverSocket.sendto(self.encrypt("Invalid Command"), addr)
        elif c == "SET_DEGF" :
            self.deg = "F"
        elif c == "SET_DEGC" :
            self.deg = "C"
        elif c == "SET_DEGK" :
            self.deg = "K"
        elif c == "GET_TEMP" :
            self.serverSocket.sendto(self.encrypt("%f" % self.getTemperature()), addr)
        elif c == "UPDATE_TEMP" :
            self.updateTemperature()
        elif c :
            self.serverSocket.sendto(self.encrypt("Invalid Command"), addr)
```

So, to patch this vulnerability, I split the command by semicolon and check if the length is not equal to 1 (one command) then it return “Invalid Command”. This ensures that exactly one command is processed per authenticated request, preventing command chaining attacks.

```
def processCommands(self, msg, addr) :
    ## Split commands by semicolon
    cmds = [c for c in msg.split(';') if c]
    if len(cmds) != 1:
        self.serverSocket.sendto(self.encrypt("Invalid Command"), addr)
        return
```

The full code for **processCommands()** function:

```
def processCommands(self, msg, addr) :
    ## Split commands by semicolon
    cmds = [c for c in msg.split(';') if c]
    if len(cmds) != 1:
        self.serverSocket.sendto(self.encrypt("Invalid Command"), addr)
        return

    else :
        c = cmds[0]
        cs = c.split(' ')
        if len(cs) == 2 : #should be either AUTH or LOGOUT
            if cs[0] == "AUTH":
                self.delExpiredToken() ##We check and delete expired token every time we need to authenticate
                if cs[1] == SERVER_PASSWORD :
                    if len(self.tokens) >= MAX_TOKENS :
                      oldest_token = min(self.tokens, key=self.tokens.get) ## Delete the oldest token if the token list is full
                      del self.tokens[oldest_token]
                    newToken=''.join(random.choice(string.ascii_uppercase + string.ascii_lowercase + string.digits) for _ in range(16))
                    self.tokens[newToken] = time.time() ## Add the new token to the token list
                    self.serverSocket.sendto(self.encrypt(newToken), addr)
                    #print (self.tokens[-1])
                else:
                    self.serverSocket.sendto(self.encrypt("Incorrect password"), addr)
            elif cs[0] == "LOGOUT":
                if cs[1] in self.tokens :
                    del self.tokens[cs[1]]
            else : #unknown command
                self.serverSocket.sendto(self.encrypt("Invalid Command"), addr)
        elif c == "SET_DEGF" :
            self.deg = "F"
        elif c == "SET_DEGC" :
            self.deg = "C"
        elif c == "SET_DEGK" :
            self.deg = "K"
        elif c == "GET_TEMP" :
            self.serverSocket.sendto(self.encrypt("%f" % self.getTemperature()), addr)
        elif c == "UPDATE_TEMP" :
            self.updateTemperature()
        elif c :
            self.serverSocket.sendto(self.encrypt("Invalid Command"), addr)
```